Last Resort Hotel Management System - Web Analysis

Design Overview

Project Context

Last Resort is a comprehensive hotel management system designed to handle the complex operations of a luxury hotel. The web application serves as the administrative interface for hotel staff to manage reservations, room inventory, guest parties, events, billing, and maintenance operations.

Core Application Modules

The system consists of 8 primary operational modules:

1. **Dashboard** - Real-time operational metrics and KPIs
2. **Inventory** - Room configuration and availability tracking
3. **Parties** - Guest group management and contact information
4. **Reservations** - Booking lifecycle management
5. **Assignments & Stay** - Room assignments and check-in/check-out tracking
6. **Events** - Event scheduling and multi-room event management
7. **Billing** - Invoice and charge tracking
8. **Maintenance** - Issue tracking and resolution workflows

Page-by-Page Design Analysis

1. Dashboard

Purpose: Executive overview and operational intelligence

Key Features:

- **Dynamic Filtering System**: Multi-dimensional filtering by date range, building, wing, and party type
- **KPI Cards**: Real-time metrics display (total revenue, active assignments, events count, average stay duration, open maintenance tickets)
- **Occupancy Table**: Room status breakdown by building showing available, occupied, and maintenance counts
- **Hierarchical Filtering Logic**: JavaScript-driven cascade where building selection dynamically filters available wings

User Flows:

1. View dashboard with default filters
2. Apply filters to narrow operational scope
3. Monitor KPI trends and occupancy patterns
4. Clear filters to reset view

Data Structure:

```
Dashboard displays aggregated data from:
- reservation → occupancy metrics
- room → status tracking
- stay → active check-ins
- event → count aggregations
- maintenance_ticket → open issues
```

Interactive Elements:

- Date pickers for temporal filtering
- Dropdowns for hierarchical property selection
- Button controls for apply/clear actions

---

2. Inventory Management

Purpose: Room configuration, facility features, and availability state

Key Features:

- **Room Search & Filtering**: Search by room ID, building, floor, status
- **Room Details**: Display room capacity, base rate, and current status
- **Bed Configuration**: View and edit bed types and quantities per room
- **Functional Assignments**: Track which services/functions each room supports (conference, dining, etc.)
- **Status Management**: Update room status (available, occupied, maintenance)

User Flows:

1. Search for specific room
2. View room features and bed configurations
3. Update room status or assign new functions
4. Filter rooms by availability or building location

Data Structure:

```
Room hierarchy:
- building → floor → room (one-to-many)
- room ↔ bed_type (many-to-many via room_has_bed)
- room ↔ function (many-to-many via room_function)
- room → maintenance_ticket (issue tracking)
```

---

3. Parties & Guest Management

Purpose: Contact management and guest group administration

Key Features:

- **Party Search**: Find existing parties by name, email, or phone
- **Party Creation**: Add new individual or organizational parties
- **Contact Details**: Store and edit contact information (name, email, phone)
- **Party Type Classification**: Distinguish between individual guests and organizations
- **Guest Group Membership**: Track which guests belong to each party

User Flows:

1. Search for existing party or create new
2. Enter contact and party type information
3. Add guest members to party group
4. Link party to reservations

Data Structure:

```
party (contact entity)
  └─ guest_group (membership tracking)
  └─ billing_account (credit/payment account)
  └─ reservation (bookings)
  └─ event (hosted events)
```

---

4. Reservations

Purpose: Booking lifecycle management with multi-step creation

Design Pattern: **Two-Mode Interface**

- **View Mode**: Search and update existing reservations
- **Create Mode**: Wizard-based new booking process

View Mode Features:

- Search reservations by contact name
- Display reservation date ranges and current status
- Inline status updates (Confirmed, Pending, Completed, Cancelled)

Create Mode - Step-by-Step Process:

Step 1: Party Selection

- Dual-panel interface: existing party search + new party creation
- Search results show matching contacts
- Quick selection to proceed or create new party form
- Related guest groups displayed for context

Step 2: Guest Group Addition

- Add individual guest names to party group
- Display current group membership

- Button to proceed when group is complete

Step 3: Finalization

- Set reservation dates (start/end)
- Specify status (Confirmed/Pending)
- Enter deposit amount
- Auto-populated date created field

User Flows:

1. **View & Update**: Search → Select reservation → Change status
2. **New Booking**: Select party → Add guests → Define dates & deposit → Confirm

Data Relationships:

```
reservation (parent)
  └─ party (contact)
  └─ guest_group (guest members)
  └─ stay (actual check-in sessions)
  └─ room_assignment (room allocation)
```

---

5. Assignments & Stay

Purpose: Bridge reservations to actual room occupancy and track guest stays

Key Features:

- **Room Assignment**: Allocate specific rooms to reservations
- **Check-in/Check-out**: Manage stay timing and transitions
- **Stay Tracking**: Record actual occupancy periods within a reservation
- **Multi-Stay Support**: Support multiple stay periods per reservation

User Flows:

1. Select reservation from list
2. Assign available rooms to reservation
3. Record check-in time
4. Record check-out time (when guest leaves)
5. Track stay status through completion

Data Structure:

```
reservation
  └─ stay (check-in session)
      └─ room_assignment (room-specific assignment)
          └─ room (physical room)
```

---

6. Events

Purpose: Event hosting and multi-room event management

Key Features:

- **Event Creation**: Define event type, time, and expected guest count
- **Host Party**: Link event to hosting party
- **Multi-Room Assignment**: Allocate multiple rooms for single event via event_room junction table
- **Event Type Classification**: Categorize events (conference, wedding, gala, etc.)

User Flows:

1. Create new event with host party and details
2. Specify start/end times and estimated attendance
3. Assign one or more rooms to event
4. Track event status and room utilization

Data Structure:

```
event (hosted by party)
  ├─ party (host organization/individual)
  └─ event_room (multi-room allocation)
      └─ room (venue spaces)
```

---

7. Billing & Charges

Purpose: Revenue tracking, charge management, and account management

Key Features:

- **Billing Accounts**: Maintain credit limits and account status per party
- **Charge Recording**: Track charges against accounts
- **Service-Based Billing**: Apply service type rates (room rate, services, etc.)
- **Stay-Linked Charges**: Associate charges with specific stays
- **Account Status Management**: Track account status (active, suspended, etc.)

User Flows:

1. View billing account for party
2. Record new charge (service + amount)
3. Track charges by stay period
4. Monitor account credit limit and status
5. Generate billing summary

Data Structure:

```
billing_account (party account)
  └─ charge (individual line items)
      ├─ service_type (service classification)
      └─ stay (occupancy period association)
```

---

8. Maintenance

Purpose: Issue tracking and resolution workflow

Key Features:

- **Ticket Creation**: Report maintenance issues for specific rooms
- **Issue Description**: Detailed problem documentation
- **Status Tracking**: Track ticket through lifecycle (Open, In Progress, Completed)
- **Resolution Recording**: Document resolution date
- **Room Status Integration**: Automatic room status update to "Maintenance" when open tickets exist

User Flows:

1. Create maintenance ticket for room
2. Describe issue and set priority
3. Track ticket status updates
4. Record resolution and completion date
5. Verify room status updates automatically

Data Structure:

```
maintenance_ticket
  └─ room (affected room)
     └─ currentStatus (automatically set to "Maintenance")
```

Summary

The Last Resort web application demonstrates **pragmatic architecture choices** optimized for:

- **User Efficiency**: Reduced clicks, guided workflows, progressive disclosure
- **Data Integrity**: Cascade filters, automatic status updates, foreign key constraints
- **Maintainability**: Server-side rendering, normalized database, clear module separation
- **Flexibility**: Optional fields, many-to-many relationships, extensible event system

These design decisions prioritize **operational usability** over technical sophistication, making the system valuable for hotel staff while remaining simple to deploy and maintain.